

Interface Inteligente para Consulta de Arquivos CSV

Relatório Técnico -- Desafio 4

I2A2 -- Instituto de Inteligência Artificial Aplicada

Agentes Inteligentes e LLMs

Framework de agentes: LangChain (create_agent / LangGraph)

Modelo de linguagem: Google Gemini (gemini-2.5-flash)

Interface: Streamlit

1. Framework escolhido

O agente foi construído com **LangChain**, usando a API atual de agentes da versão 1.x (`langchain.agents.create_agent`), que orquestra o loop de tool-calling internamente via LangGraph. O modelo de linguagem é o **Google Gemini 2.5 Flash** (via `langchain-google-genai`), escolhido por ter um tier gratuito generoso -- adequado para um protótipo de curso testado por vários integrantes do grupo sem custo. A interface das duas etapas (carga de dados e consulta) foi feita em **Streamlit**, o que também permite publicação gratuita no Streamlit Community Cloud.

Observação técnica: a API legada do LangChain (`AgentExecutor + create_tool_calling_agent`) foi removida na versão 1.x instalada neste projeto -- o código foi adaptado para a API atual (`create_agent`), que é a forma recomendada hoje e já é construída sobre LangGraph.

2. Arquitetura da solução

A aplicação é dividida em módulos de responsabilidade única, sem lógica de negócio misturada na interface:

Arquivo	Responsabilidade
<code>app.py</code>	Interface Streamlit: Aba A (upload) e Aba B (chat)
<code>src/csvagent/ingestion.py</code>	Extrai o ZIP; detecta encoding e separador por arquivo; identifica o dicionário de dados
<code>src/csvagent/catalog.py</code>	Catálogo em memória: DataFrames + schema + dicionário de dados; gera o resumo textual para o prompt
<code>src/csvagent/sandbox.py</code>	Execução restrita (whitelist de AST) do código pandas/plotly gerado pelo agente
<code>src/csvagent/tools.py</code>	5 tools do agente + registro do trace de decisão para a UI
<code>src/csvagent/agent.py</code>	Monta o agente: system prompt + LLM (Gemini) + tools

Interface A -- Carga dos dados

O usuário envia um .zip com um ou mais CSVs e, opcionalmente, um dicionário de dados. O pipeline de ingestão: (1) extrai os membros do zip ignorando lixo de SO (`__MACOSX`, arquivos ocultos); (2) para cada arquivo .csv/.txt, detecta o encoding via `chardet` e o separador via `csv.Sniffer` com fallback manual (testa `;` e `\t`); (3) classifica cada arquivo como DADOS ou DICIONÁRIO DE DADOS por heurística de nome (ex: "dicionario", "layout", "metadados") e, na ausência disso, pelo formato do cabeçalho (poucas colunas com nomes como "coluna"/"descrição"); (4) monta um catálogo em memória com todas as tabelas e o dicionário associado. Isso resolve, de forma automática, os dois formatos de exemplo do curso (NFs e NFe), que trazem encoding e separador diferentes dos padrões americanos.

Interface B -- Consulta em linguagem natural

O catálogo (schema + amostra + dicionário) é injetado no system prompt do agente. A cada pergunta do usuário, o agente decide livremente qual sequência de tools chamar. O resultado de cada chamada

de tool é registrado (nome da tool, código gerado, resultado real) e exibido num expander "Ver raciocínio do agente" -- é essa trilha que evidencia, na própria interface, como o agente chegou à resposta.

Sandbox de execução

O código pandas/plotly escrito pelo LLM não é executado livremente: passa antes por um validador de AST (`sandbox.py`) que bloqueia `import`, `eval/exec`, acesso a atributos `__dunder__`, definição de função/classe e loops `while`. Os únicos nomes expostos ao código são `pd`, `px/go` e o dicionário `tables` com cópias dos DataFrames carregados. Não é isolamento de processo/SO -- é uma defesa em profundidade adequada ao escopo educacional do desafio (ver seção de limitações).

3. Descrição do agente e das ferramentas (tools)

A solução usa um único agente (arquitetura enxuta, conforme recomendado pelo enunciado: "não esperamos utilização obrigatória de múltiplos agentes"), equipado com 5 tools especializadas:

- **listar_tabelas** -- Lista as tabelas carregadas, com número de linhas e colunas. Usada quando o agente não tem certeza do nome exato da tabela.
- **descrever_tabela** -- Mostra schema detalhado (nome/tipo de cada coluna + descrição do dicionário de dados), estatísticas de colunas numéricas e uma amostra de linhas.
- **valores_frequentes** -- Retorna os valores mais frequentes de uma coluna categórica, com contagem -- útil para explorar categorias antes de filtrar/agrupar.
- **executar_pandas** -- Executa uma consulta pandas real sobre as tabelas (agregações, somas, contagens, rankings, filtros) e retorna o resultado real, nunca um número "chutado".
- **gerar_grafico** -- Gera um gráfico Plotly (`px/go`) a partir das tabelas, quando o usuário pede um gráfico ou quando uma visualização ajuda a resposta.

O system prompt instrui explicitamente o modelo a nunca responder de memória: toda resposta quantitativa deve vir da execução real de uma tool. Isso é o que diferencia esta solução de "perguntar direto para uma LLM genérica" -- vedado pelo enunciado do desafio -- já que os números apresentados vêm sempre da execução de código sobre os dados efetivamente carregados na sessão.

4. Fluxo de funcionamento da aplicação

- 1 Usuário envia um .zip na Aba A.
- 2 `ingestion.py` extrai, detecta encoding/separador e classifica cada arquivo (dado vs. dicionário).
- 3 `catalog.py` monta o catálogo em memória e o resumo textual do schema.
- 4 Ao abrir a Aba B, `agent.py` monta o agente: system prompt (catálogo) + LLM (Gemini) + as 5 tools.
- 5 Usuário faz uma pergunta em linguagem natural no chat.
- 6 O agente (LangGraph por baixo) decide, em loop, quais tools chamar -- cada chamada roda de fato sobre os dados (via `sandbox.py`) e o resultado real volta para o agente.

- 7 O agente produz uma resposta final em português, com números concretos.
- 8 A interface exibe a resposta, qualquer tabela/gráfico gerado, e um expander com o trace completo de tools chamadas (transparência do raciocínio).

5. Perguntas realizadas e respostas obtidas

Testes realizados com um CSV sintético de notas fiscais (5 linhas: id_nf, fornecedor, produto, categoria, valor, data_emissao) acompanhado de um dicionário de dados no mesmo .zip, processado end-to-end pela aplicação real (upload -> ingestão -> agente -> resposta). Para cada pergunta, o trace real do agente (tool chamada + código executado + resultado) é mostrado abaixo do texto -- exatamente o que aparece no expander "Ver raciocínio do agente" da Interface B.

P1. Qual fornecedor recebeu o maior valor no período?

Resposta do agente: O fornecedor que recebeu o maior valor no período foi o Fornecedor A.

tool: executar_pandas

```
resultado = tables["notas"].groupby("fornecedor")["valor"].sum().sort_values(ascending=False).head(5)
```

Resultado real retornado pela tool: Fornecedor A: 601.25 | Fornecedor B: 330.00 | Fornecedor C: 150.25

P2. Qual produto apresentou o maior volume comprado (em quantidade de notas)?

Resposta do agente: O produto "Parafuso" apresentou o maior volume comprado, com 2 notas fiscais.

tool: executar_pandas

```
resultado = tables["notas"].groupby("produto")["id_nf"].count().sort_values(ascending=False).head(1)
```

Resultado real retornado pela tool: Parafuso: 2

P3. Qual foi o total gasto em cada mês?

Resposta do agente: O total gasto em cada mês foi: Mês 1: R\$ 350,50 -- Mês 2: R\$ 651,00 -- Mês 3: R\$ 80,00.

tool: executar_pandas

```
tables["notas"]["data_emissao"] = pd.to_datetime(tables["notas"]["data_emissao"])
tables["notas"]["mes"] = tables["notas"]["data_emissao"].dt.month
resultado = tables["notas"].groupby("mes")["valor"].sum().reset_index()
```

Resultado real retornado pela tool: mes=1: 350.5 | mes=2: 651.0 | mes=3: 80.0

P4. Quais foram os cinco maiores fornecedores?

Resposta do agente: Os cinco maiores fornecedores foram: Fornecedor A (R\$ 601,25), Fornecedor B (R\$ 330,00), Fornecedor C (R\$ 150,25).

tool: executar_pandas

```
resultado = tables["notas"].groupby("fornecedor")["valor"].sum().sort_values(ascending=False).head(5)
```

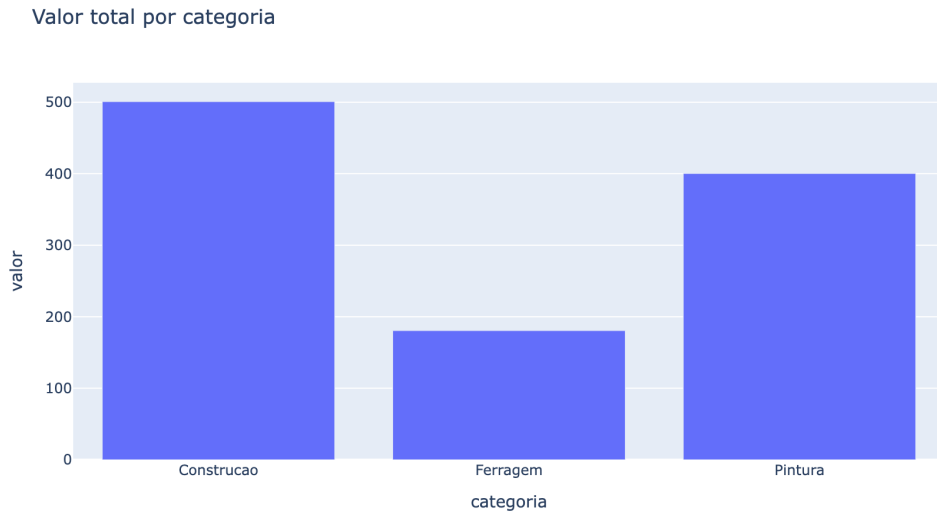
Resultado real retornado pela tool: Fornecedor A: 601.25 | Fornecedor B: 330.00 | Fornecedor C: 150.25

P5. Gere um gráfico do valor total por categoria de produto.

Resposta do agente: "Um gráfico do valor total por categoria foi gerado." (a tool gerar_grafico foi chamada com sucesso; abaixo, a renderização real produzida pelo mesmo pipeline de sandbox usado pela tool).

tool: gerar_grafico

```
agg = tables["notas"].groupby("categoria")["valor"].sum().reset_index()
fig = px.bar(agg, x="categoria", y="valor", title="Valor total por categoria")
```



6. Limitações conhecidas (MVP)

- Um único dicionário de dados é aplicado a todas as tabelas do ZIP (cenário mais comum quando cabeçalho/itens de uma mesma nota compartilham glossário); não há suporte a um dicionário por tabela.
- O sandbox impede `while` e algumas funções perigosas, mas não há timeout de execução -- um `for` sobre uma tabela muito grande ainda pode ser lento.
- Sem suporte a múltiplos ZIPs na mesma sessão (recarregar substitui o catálogo carregado).
- A API gratuita do Gemini tem cota diária (20 requisições/dia no tier free do modelo usado nos testes) -- para uso contínuo em sala de aula, recomenda-se cada integrante usar sua própria chave.

7. Como executar

```
uv sync
# criar .env com GOOGLE_API_KEY=... (ou colar a chave na barra lateral do app)
uv run streamlit run app.py
```